

Arduino Cookbook : De la Théorie à la Pratique

Chapitre 18 : Utilisation de Puces Contrôleurs

Présenté par : Dr B. DIOP

29 mars 2026

Référence : Michael Margolis, *Arduino Cookbook*, 2nd Edition, O'Reilly

18.0 Introduction : Repousser les limites physiques

Le problème du nombre de broches

L'Arduino Uno ne possède que 14 broches numériques et 6 broches analogiques. Que faire si l'on souhaite contrôler un cube de 512 LEDs ou lire l'état de 32 boutons-poussoirs simultanément ?

Dans ce chapitre, nous allons apprendre à :

- Utiliser des **Registres à Décalage (Shift Registers)** pour multiplier les sorties numériques (74HC595).
- Multiplier les entrées numériques (74HC165 / CD4021B).
- Utiliser des **Multiplexeurs** pour lire plusieurs capteurs analogiques sur une seule broche (CD4051).
- Piloter des matrices de LEDs avec un circuit dédié (MAX7219).
- Étendre les sorties PWM pour contrôler plusieurs servomoteurs (PCA9685).

18.1 Le Registre à Décalage (Shift Register 74HC595)

Le composant roi de l'extension numérique : Le 74HC595 est une puce bon marché (SIPO : Serial-In, Parallel-Out) qui permet de contrôler **8 sorties numériques** en n'utilisant que **3 broches** de l'Arduino.

Le principe de fonctionnement :

1. L'Arduino envoie un octet (8 bits, ex : 10110010) de manière sérielle, bit par bit.
2. La puce stocke ces 8 bits en mémoire interne.
3. Sur un signal de validation (Latch), la puce applique ces 8 bits simultanément sur ses 8 broches physiques de sortie (Q0 à Q7).

18.2 Câblage du 74HC595

La puce possède 16 broches. Les 3 broches de contrôle essentielles à relier à l'Arduino sont :

- **DS (Data Serial) - Broche 14** : Reçoit les bits (0 ou 1) un par un.
- **SH_CP (Shift Clock) - Broche 11** : L'horloge. À chaque impulsion, la puce lit le bit présent sur DS et décale les précédents d'une case.
- **ST_CP (Storage / Latch) - Broche 12** : Le verrou. Quand on le passe de LOW à HIGH, les états mémorisés sont envoyés vers les sorties physiques (LEDs).

Gestion de l'alimentation

Le 74HC595 ne peut fournir qu'environ 35mA au total. C'est suffisant pour quelques LEDs avec résistances, mais pas pour des moteurs ! Pour la puissance, il faut ajouter des transistors.

18.3 Code : Contrôler le 74HC595 (shiftOut)

L'Arduino possède une fonction native optimisée pour ce composant : `shiftOut()`.

```
const int dataPin  = 11; // DS
const int clockPin = 12; // SH_CP
const int latchPin = 8;  // ST_CP

void setup() {
  pinMode(dataPin, OUTPUT);
  pinMode(clockPin, OUTPUT);
  pinMode(latchPin, OUTPUT);
}

void loop() {
  byte motif = B10101010; // 1 = LED allumee, 0 = LED eteinte

  digitalWrite(latchPin, LOW); // Bloque les sorties pendant le transfert

  // Envoie l'octet bit par bit, le bit de poids fort (MSB) en premier
  shiftOut(dataPin, clockPin, MSBFIRST, motif);

  digitalWrite(latchPin, HIGH); // Valide et allume les LEDs !
  delay(1000);
}
```

18.4 Le Chaînage (Daisy-Chaining)

La véritable magie du 74HC595 est qu'il possède une broche de débordement (**Q7S - Broche 9**).

Si vous lui envoyez 16 bits au lieu de 8, les 8 premiers bits vont "pousser" et sortir par la broche Q7S.

Conséquence : En reliant la broche Q7S du premier composant à la broche DS du deuxième, vous pouvez contrôler **16, 24, 32 LEDs ou plus...** en utilisant toujours **les mêmes 3 broches** de l'Arduino ! Il suffira d'appeler la fonction `shiftOut()` plusieurs fois d'affilée avant de fermer le "Latch".

18.5 Le Registre d'Entrée (Shift Register 74HC165)

À l'inverse du 595, le 74HC165 (ou CD4021B) est un composant PISO (Parallel-In, Serial-Out).

Son utilité : Lire l'état de **8 boutons-poussoirs** simultanément et renvoyer le résultat à l'Arduino sous la forme d'un seul octet, via seulement 3 ou 4 fils.

Le principe :

1. L'Arduino envoie un signal (Latch) demandant à la puce de "prendre une photo" instantanée de ses 8 entrées matérielles.
2. L'Arduino utilise la fonction `shiftIn()` pour rapatrier cette "photo" sous forme d'un octet (ex : 00000001 si le bouton n°0 est pressé).

18.6 Code : Lire le 74HC165 (shiftIn)

```
const int dataPin  = 11; // Q7
const int clockPin = 12; // CP
const int latchPin = 8;  // PL (Parallel Load)

void setup() {
  Serial.begin(9600);
  pinMode(dataPin, INPUT);
  pinMode(clockPin, OUTPUT);
  pinMode(latchPin, OUTPUT);
}

void loop() {
  // Prend une "photo" des 8 broches
  digitalWrite(latchPin, LOW); delayMicroseconds(5);
  digitalWrite(latchPin, HIGH); delayMicroseconds(5);

  // Rapatrie la donnee
  byte etatBoutons = shiftIn(dataPin, clockPin, MSBFIRST);

  Serial.println(etatBoutons, BIN); // Affiche l'octet reçu en binaire
  delay(100);
}
```

18.7 Les Multiplexeurs Analogiques (CD4051)

Les registres à décalage ne gèrent que du numérique (0V ou 5V). Mais que faire si l'on veut lire **8 potentiomètres** et que l'Arduino Uno n'a que 6 entrées analogiques ?

Le Multiplexeur (ex : 74HC4051 ou CD4051) : C'est l'équivalent électronique d'un aiguillage de train. Il possède :

- **8 canaux indépendants (Y0 à Y7)** connectés à vos capteurs.
- **1 broche commune (Z)** connectée à la broche A0 de l'Arduino.
- **3 broches de sélection (S0, S1, S2)** connectées aux broches numériques de l'Arduino.

18.8 Fonctionnement du Multiplexeur

Le composant utilise le code binaire envoyé sur ses 3 broches de sélection pour savoir quel capteur relier physiquement à l'Arduino.

S2 (Poids fort)	S1	S0 (Poids faible)	Canal connecté à A0
0	0	0	Y0 (Capteur 1)
0	0	1	Y1 (Capteur 2)
0	1	0	Y2 (Capteur 3)
...	...	...	...
1	1	1	Y7 (Capteur 8)

Note : Les multiplexeurs sont bidirectionnels (Demultiplexeur). Ils peuvent aussi distribuer un seul signal PWM vers 8 moteurs différents !

18.9 Code : Lire 8 capteurs via le CD4051

L'Arduino doit boucler rapidement sur les combinaisons binaires (de 0 à 7) et lire la broche A0 à chaque changement.

```
const int pinS0 = 2; const int pinS1 = 3; const int pinS2 = 4;
const int analogPin = A0;

void setup() {
  Serial.begin(9600);
  pinMode(pinS0, OUTPUT); pinMode(pinS1, OUTPUT); pinMode(pinS2, OUTPUT);
}

void loop() {
  for (int i = 0; i < 8; i++) {
    // Selectionne le canal en écrivant les bits de 'i' sur S0, S1, S2
    digitalWrite(pinS0, bitRead(i, 0));
    digitalWrite(pinS1, bitRead(i, 1));
    digitalWrite(pinS2, bitRead(i, 2));

    // Attend que le signal se stabilise
    delay(1);

    int valeurCapteur = analogRead(analogPin); // Lit le capteur selectionne
    Serial.print(valeurCapteur); Serial.print("\t");
  }
}
```

18.10 Pilotes de Matrices LED : Le MAX7219

Piloter une matrice de LEDs 8×8 (64 LEDs) avec des registres 74HC595 classiques est complexe : il faut gérer la persistance rétinienne et le balayage de l'écran dans le code Arduino, ce qui sature le processeur.

La puce MAX7219 : C'est un "Display Driver" autonome. Vous lui envoyez l'image à afficher (via le bus SPI), et la puce gère elle-même le rafraîchissement des 64 LEDs en tâche de fond !

Avantages :

- Gère les matrices 8×8 ou les afficheurs 7 segments (jusqu'à 8 chiffres).
- Permet de régler la luminosité de manière logicielle.
- Se chaîne très facilement pour créer de grands panneaux défilants.

18.11 Code : Utilisation du MAX7219

On utilise généralement la bibliothèque tierce `LedControl.h` pour simplifier l'envoi des commandes.

```
#include <LedControl.h>

// Broches : DIN = 12, CLK = 11, CS = 10, Nombre de modules = 1
LedControl lc = LedControl(12, 11, 10, 1);

void setup() {
    lc.shutdown(0, false); // Reveille le MAX7219
    lc.setIntensity(0, 8); // Regle la luminosite (0 a 15)
    lc.clearDisplay(0);    // Efface l'ecran
}

void loop() {
    // Allume la LED a la coordonnee (Ligne 3, Colonne 4) du module n 0
    lc.setLed(0, 3, 4, true);
    delay(1000);
    lc.setLed(0, 3, 4, false); // Eteint la LED
    delay(1000);
}
```

18.12 Étendre le PWM : Le contrôleur PCA9685

L'Arduino Uno possède 6 sorties PWM (pour contrôler la vitesse des moteurs ou les servomoteurs). Et si l'on construit un robot-araignée nécessitant 18 servomoteurs ?

Le contrôleur PCA9685 :

- Géré via le **bus I2C** (SDA/SCL, seulement 2 fils nécessaires!).
- Génère **16 canaux PWM matériels** complètement indépendants de l'Arduino.
- Résolution de 12 bits (4096 niveaux de précision), bien supérieure au PWM 8 bits (256 niveaux) de l'Arduino.
- Possède sa propre horloge interne. L'Arduino envoie un ordre et peut faire autre chose pendant que la puce maintient la position des moteurs.

18.13 L'importance des Alimentations Séparées

Règle de sécurité en électronique étendue

Dès que vous commencez à utiliser des contrôleurs de puces pour piloter plusieurs composants matériels (matrices LED, multiples servomoteurs via PCA9685), le régulateur 5V interne de l'Arduino ne suffit plus !

Surchauffe et Reset : Si vous tirez plus de 500mA depuis la broche 5V de l'Arduino (connectée en USB), la carte plantera ou le fusible réarmable sautera.

Solution : Il faut fournir une alimentation externe dédiée (ex : transformateur 5V 3A) directement aux puces contrôleurs, et relier impérativement la broche GND (Masse) de cette alimentation à la broche GND de l'Arduino.

- **74HC595 (SIPO)** : Le "Couteau Suisse" pour multiplier les sorties numériques (LEDs, relais) via `shiftOut()`. Se chaîne à l'infini.
- **CD4051 (Multiplexeur)** : L'aiguillage parfait pour lire plusieurs potentiomètres ou capteurs sur une seule broche analogique de l'Arduino.
- **Puces Intelligentes** : Les puces comme le MAX7219 (LEDs) ou le PCA9685 (PWM) soulagent le processeur de l'Arduino en gérant le travail répétitif de manière matérielle autonome.
- **Puissance** : Une puce d'extension ne fait que router des signaux "logiques". Pour piloter de la puissance, il faudra toujours l'associer à des transistors, MOSFET ou relais.

Fin du Chapitre 18 et clôture de cette formation académique basée sur l'Arduino Cookbook !